# Amrita School of Computing

Amritapuri Campus

Department of Computer Science and Engineering

**B.Tech Computer Science and Engineering**

**23CSE498 – Project Phase II**

## Design and Solution Submission

**Project Title** Retrieval Beyond Text: Designing a Context-Aware Retrieval Framework for Multi-Party, Time-Dependent Conversations

**Project Guide** Ms. Bindhya Bhadran

### Team Members

| Team No. | Roll Number | Name of Student |
|---|---|---|
| B10 | AM.SC.U4CSE23130 | K. Sai Dinesh |
| B10 | AM.SC.U4CSE23150 | Satya Sri Dheeraj Motupalli |
| B10 | AM.SC.U4CSE23171 | Y. Sai Nikhil |

Academic Year: 2026–2027

**Amrita Vishwa Vidyapeetham**

# 1 Sustainable Development Goal (SDG) Mapping

**Primary SDG:** SDG 9 – Industry, Innovation and Infrastructure

**Justification (100–150 words):** This project contributes to **SDG 9** by developing a novel, privacy-conscious retrieval infrastructure for a class of data (fragmented, multi-party, time-dependent conversational text) that existing RAG frameworks and memory architectures were not designed to handle. It advances applied AI infrastructure by adapting retrieval techniques – thread-aware chunking, speaker-aware context, and time-weighted ranking – to a widely-used but underserved data modality, and validates the approach through empirical benchmarking rather than relying on existing tools alone.

| SDG | Goal |
|---|---|
| 1 | No Poverty |
| 2 | Zero Hunger |
| 3 | Good Health and Well-being |
| 4 | Quality Education |
| 5 | Gender Equality |
| 6 | Clean Water and Sanitation |
| 7 | Affordable and Clean Energy |
| 8 | Decent Work and Economic Growth |
| 9 | Industry, Innovation and Infrastructure |
| 10 | Reduced Inequalities |
| 11 | Sustainable Cities and Communities |
| 12 | Responsible Consumption and Production |
| 13 | Climate Action |
| 14 | Life Below Water |
| 15 | Life on Land |
| 16 | Peace, Justice and Strong Institutions |
| 17 | Partnerships for the Goals |

# 2 Project Milestones and Timeline

| Semester | Major Activity / Milestone | Expected Deliverable | Expected Completion | Status |
|---|---|---|---|---|
| S6 | Project domain exploration | Problem domain identified | 15 Feb 2026 | Completed |
| S6 | Literature review | Initial literature survey | 15 Mar 2026 | Completed |
| S6 | Baseline implementation | Reference implementation completed | 30 Apr 2026 | Completed |
| S7 (Submission 1) | Literature survey | Comprehensive literature survey | 15 Jul 2026 | Completed |
| S7 (Submission 1) | Research gap identification | Research gap statement | 20 Jul 2026 | Completed |
| S7 (Submission 1) | Problem statement | Finalized objectives | 25 Jul 2026 | Completed |
| S7 (Submission 2) | System design | Architecture and workflow | 06 Aug 2026 | Current |
| S7 | Prototype implementation | Functional prototype | 15 Oct 2026 | Planned |
| S7 | Module integration | Integrated system | 15 Nov 2026 | Planned |
| S7 | Testing and validation | Experimental results | 15 Dec 2026 | Planned |
| S8 | System enhancement | Improved system | 31 Jan 2027 | Planned |
| S8 | Performance evaluation | Performance analysis | 28 Feb 2027 | Planned |
| S8 | Research publication | Paper submission | 20 Mar 2027 | Planned |
| S8 | Dissertation writing | Final report | 10 Apr 2027 | Planned |
| S8 | Final demonstration | Presentation and viva | 20 Apr 2027 | Planned |

Table 1: Overall Project Timeline

# 3 System Architecture

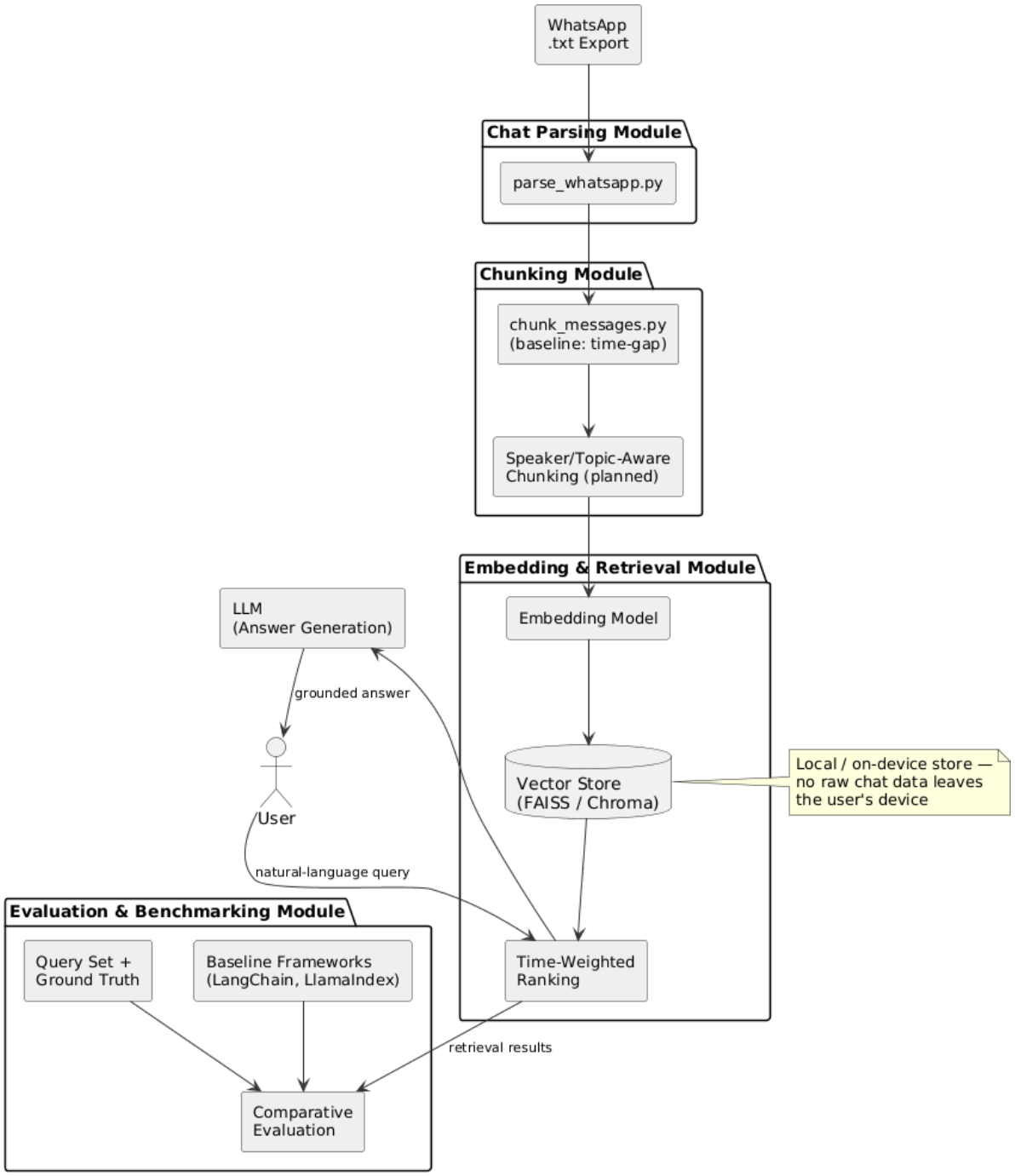



Figure 1: System Architecture

## Architecture Description

The system is organized as a linear pipeline that transforms a raw WhatsApp chat export into a queryable retrieval index, followed by a retrieval-augmented answer generation

stage:

WhatsApp Export ⟶ Parse & Structure ⟶ Chunking ⟶ Embedding + Vector Store ⟶ Time-Weighted Retrieval ⟶ LLM ⟶ Answer

The **Parsing** stage converts the raw `.txt` export into structured {date, time, sender, text} records, correctly merging multi-line/continuation messages and filtering out system notices and media-only placeholders. The **Chunking** stage groups consecutive messages into retrievable units; a naive time-gap baseline (30-minute silence threshold) has already been implemented and evaluated, and is being extended toward speaker-aware and semantic/topic-boundary-aware chunking. The **Embedding + Vector Store** stage (in progress) will convert chunks into dense vector representations for similarity search. The **Time-Weighted Retrieval** stage will rank retrieved chunks using both semantic similarity and recency/time-relevance to the query. Finally, an **LLM** synthesizes the retrieved chunks into a natural-language answer grounded in the user's own chat history. All processing is designed to run on data the user has exported themselves, with an on-device/local-processing option treated as a privacy-preserving stretch goal.

# 4 Module Description

| Module | Description |
|---|---|
| Chat Parsing Module | Converts a raw WhatsApp .txt export into structured {date, time, sender, text} records. Merges multi-line/continuation messages (which carry no timestamp of their own) into the correct parent message, and filters out system notices (e.g., encryption notices, "X added Y") and media-only placeholders (<Media omitted>). Implemented in parse_whatsapp.py. |
| Chunking Module | Groups the cleaned message stream into retrievable chunks. The current baseline groups consecutive messages while the time gap between them stays under 30 minutes. This module is being extended to incorporate speaker-aware structure and semantic/topic-boundary detection, since naive time-gap chunking was empirically found to produce highly uneven chunk sizes. Implemented in chunk_messages.py (baseline); adapted version planned. |
| Embedding & Retrieval Module | Converts chunks into dense vector embeddings, stores them in a local vector store, and retrieves the top-matching chunks for a natural-language query, ranked using both semantic similarity and time-weighting. Not yet started; embedding model and vector store choice are open decisions (FAISS vs. Chroma under consideration). |
| Evaluation & Benchmarking Module | Runs a hand-labeled query set (natural-language questions with verified ground-truth answers) against the naive baseline, existing frameworks (LangChain, LlamaIndex), and the proposed adapted retrieval approach, scoring retrieval hit-rate and answer correctness, and categorizing failure modes. |

# 5 System Workflow

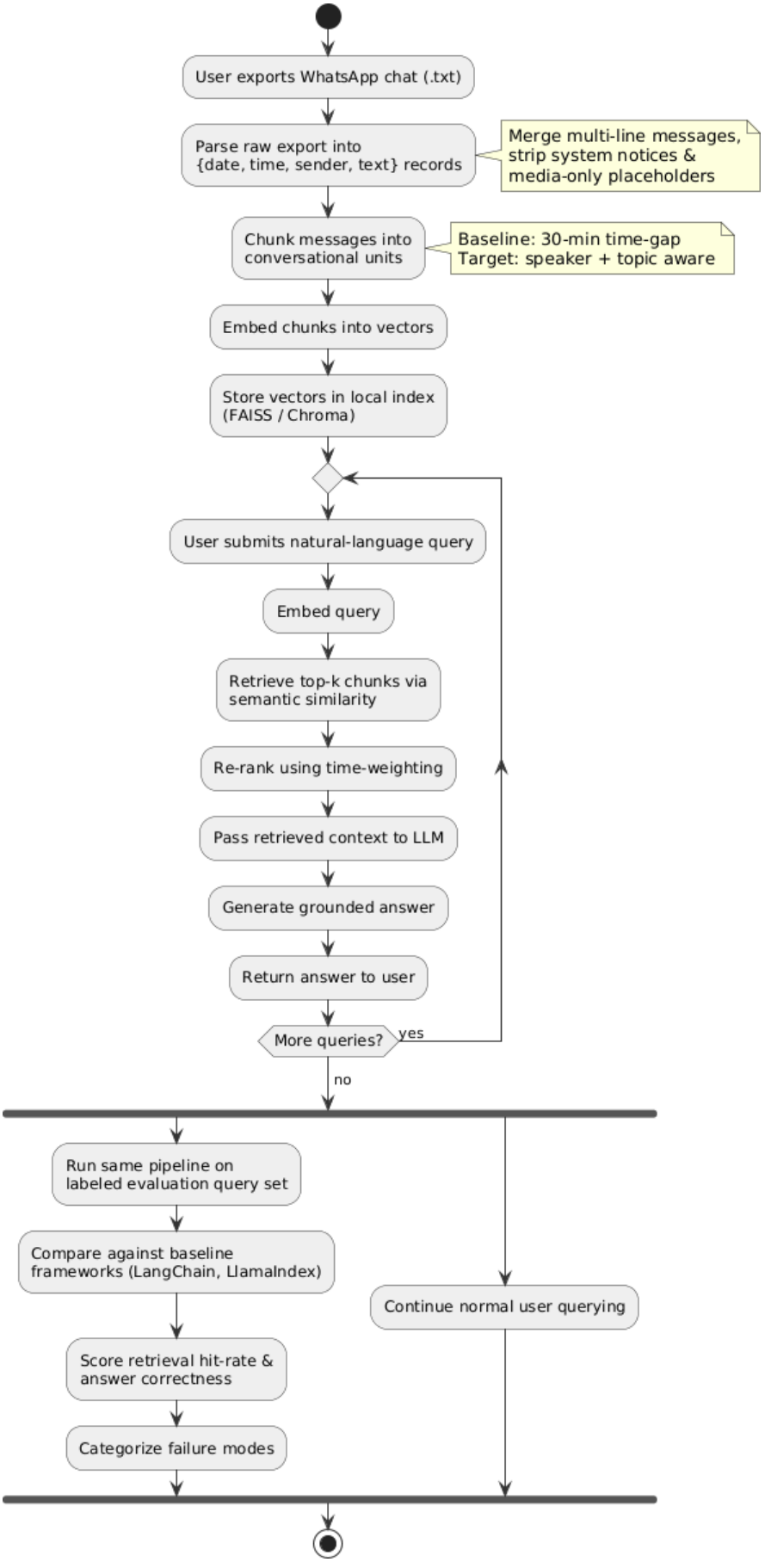



Figure 2: System Workflow

## Workflow Description

1. The user exports a WhatsApp conversation (1-on-1, or small 2–3 person group) as a .txt file and provides it to the system. 2. The Chat Parsing Module reads the export and produces a cleaned, structured JSON of {date, time, sender, text} records, discarding system messages and media-only placeholders. 3. The Chunking Module groups these records into coherent conversational units, currently via a time-gap baseline, moving toward speaker- and topic-aware grouping. 4. Each chunk is embedded and stored in a local vector index (Embedding & Retrieval Module). 5. When the user issues a natural-language query (e.g., “what did we decide about the trip?”), the system retrieves the most relevant chunks using semantic similarity combined with time-weighted ranking. 6. The retrieved chunks are passed to an LLM, which synthesizes a grounded natural-language answer referencing the original messages. 7. In parallel, the Evaluation & Benchmarking Module runs the same pipeline against a held-out labeled query set to measure retrieval accuracy and answer correctness relative to baseline RAG frameworks.

# 6 Domain-Specific Design

| Design Component | Details / Reference |
|---|---|
| **Database Schema / ER Diagram** | Not applicable in the traditional relational sense – persisted state is a JSON record schema ({date, time, sender, text}) plus a vector index (embedding → chunk mapping), rather than a relational database. |
| **UML Diagrams** | |
| **Machine Learning Model Architecture** | Retrieval-Augmented Generation (RAG) architecture: pre-trained embedding model for chunk/query vectorization + vector similarity search + time-weighted re-ranking, followed by an LLM for answer generation over retrieved context. No model is trained from scratch. |
| **Dataset Description** | Primary: self-collected WhatsApp `.txt` exports (personal/group chats, with consent). Current working file is a real 1-on-1 chat export (Aug 2023–2024+, ˜7,870 raw lines; 5,075 message blocks → 4,126 after cleaning). Supplementary: public WhatsApp datasets from prior NLP research (adapted from sentiment/topic tasks). Evaluation set: custom-built natural-language queries with manually verified ground-truth answers, since no standard retrieval-QA benchmark exists for personal chat data. |
| **API Design** | |
| **Network Topology** | |
| **Security Architecture** | |
| **Hardware Block Diagram** | |
| **Deployment Architecture** | On-device / local processing is the target deployment model (stretch goal) to avoid sending personal chat data to third-party or cloud services, directly addressing the privacy concern identified in the problem statement. |
| **Other Design Artifacts** | |

# 7 Algorithms / Methodology

## Proposed Algorithm / Methodology Description

The proposed methodology adapts standard RAG retrieval specifically for fragmented, multi-party, time-dependent conversational data, in response to gaps identified in the literature review (Section 4 of the reference document): existing frameworks and memory architectures are built either for structured documents or for single-user AI-assistant dialogue, and none jointly handle multi-speaker context, short/fragmented messages, and time-awareness together.

**1. Thread/Reply-Aware Chunking.** Since exported .txt files do not preserve WhatsApp's internal reply-quote metadata, reply relationships are approximated using semantic similarity between non-adjacent messages combined with timing proximity, rather than reconstructing the true reply graph. This is explicitly treated as an approximation, and is scoped primarily to 1-on-1 (or small 2–3 person) chats, where two-party disambiguation keeps false positives low.

**2. Speaker-Aware Context.** Each chunk retains explicit speaker attribution per message/turn, so retrieved context preserves who said what – information that naive time-gap chunking does not structurally guarantee.

**3. Semantic/Topic-Boundary Chunking.** As a refinement over the naive 30-minute time-gap baseline (which was empirically shown to produce very uneven chunks, e.g. a single 212-message chunk), chunk boundaries will additionally be informed by topic-shift detection, so that a single chunk stays semantically focused for embedding and retrieval.

**4. Time-Weighted Retrieval Ranking.** Retrieval combines semantic (embedding) similarity with a time-decay/time-relevance weighting, since conversational relevance is often tied to recency or to a specific past time window (e.g., "what did we decide last month").

**5. Evaluation Methodology.** The same custom query set (˜20–30 natural-language questions with manually verified ground-truth answers per chat) is run through (a) the naive time-gap baseline, (b) one or two existing frameworks (LangChain, LlamaIndex), and (c) the proposed adapted approach. Retrieval hit-rate and answer correctness (correct / partial / wrong) are measured, and failures are categorized (bad chunk boundary, lost speaker context, wrong time window, irrelevant retrieval) as a diagnostic contribution in its own right.

## Pseudocode / Mathematical Formulation (Optional)

```
FUNCTION build_index(raw_export):
    records  <- parse_whatsapp(raw_export)      # {date, time, sender, text}
    chunks   <- chunk_messages(records)          # time-gap / speaker / topic aware
    vectors  <- embed(chunks)
    store(vectors, chunks)                       # vector index

FUNCTION answer_query(query, top_k):
    q_vec       <- embed(query)
    candidates  <- vector_search(q_vec, top_k)
    ranked      <- rerank(candidates, sim=semantic_score, weight=time_decay(now -
    context     <- top(ranked, k=top_k)
    RETURN llm_generate(query, context)
```

Time-decay weighting example: $\text{score}(c) = \alpha \cdot \text{sim}(q, c) + (1 - \alpha) \cdot e^{-\lambda \cdot \Delta t(c)}$, where $\Delta t(c)$ is the time elapsed since chunk $c$, and $\alpha$, $\lambda$ are tunable hyperparameters to be determined empirically.

# 8 Technology Stack

| Category | Version | Technology / Tool |
|---|---|---|
| **Programming Language** | | Python |
| **Framework** | | Baseline comparison frameworks: LangChain, LlamaIndex (evaluation only) |
| **Frontend** | | |
| **Backend** | | |
| **Database** | | Vector store – FAISS vs. Chroma (decision pending) |
| **Libraries** | | Embedding model – TBD (multilingual model under consideration for code-mixed text) |
| **IDE / Development Environment** | | |
| **Version Control** | | |
| **Cloud / Deployment Platform** | | On-device / local processing (target; stretch goal) |
| **Operating System** | | |
| **Other Tools** | | |

# 9 Hardware and Software Requirements

## Hardware Requirements

| Component | Specification |
|---|---|
| **Processor** | Intel Core i5 (10th Gen) / AMD Ryzen 5 or higher – multi-core CPU recommended for parallel chunk embedding and preprocessing |
| **RAM** | 16 GB minimum (32 GB recommended) – needed to hold large chat exports, embedding batches, and a local vector index in memory simultaneously |
| **Storage** | 20 GB free SSD space – for raw/parsed chat data, vector index files, embedding model weights, and evaluation artifacts |
| **GPU (If Applicable)** | Optional – NVIDIA GPU with 6 GB+ VRAM (e.g., RTX 3060 or higher) recommended if a locally-hosted embedding/LLM model is used for on-device inference; not required if using CPU-only embedding models or API-based LLM calls |
| **Internet Connectivity** | Required during development (package installation, API-based LLM calls, literature/dataset access); not required at inference time if the on-device deployment goal is achieved |
| **Other Hardware** | Standard development laptop/desktop; no specialized hardware (sensors, IoT devices, etc.) required |

## Software Requirements

| Software | Version / Details |
|---|---|
| **Operating System** | Windows 10/11, macOS, or Ubuntu 20.04+ (Linux recommended for compatibility with ML/embedding libraries) |
| **Programming Language(s)** | Python 3.10+ |
| **Framework(s)** | LangChain and LlamaIndex (for baseline comparison); Sentence-Transformers or equivalent for embedding generation |
| **Database** | Vector store – FAISS or ChromaDB (local, no server required) – TBD |
| **IDE / Editor** | VS Code / PyCharm / Jupyter Notebook |
| **Libraries / Packages** | pandas, numpy, sentence-transformers (or chosen embedding library), faiss-cpu / chromadb, langchain, llama-index, scikit-learn (for evaluation metrics) |
| **Other Dependencies** | Git for version control; access to an LLM API (e.g., for answer generation) or a locally-hosted open-source LLM if the on-device stretch goal is pursued |

# 10 Design Considerations (Non-Functional)

| Design Aspect | Description |
|---|---|
| **Scalability** | The pipeline must handle chat histories accumulating over months or years, and the naive chunking baseline has already exposed scalability issues (e.g., a single 212-message chunk from an uninterrupted conversation), motivating semantic/topic-boundary-aware chunking. |
| **Performance** | Retrieval must return relevant chunks quickly enough for interactive natural-language querying; latency is tracked as an optional evaluation metric, particularly relevant if the on-device deployment angle is pursued. |
| **Security** | Chat data is processed from user-exported files rather than live account access, limiting exposure; no third-party/cloud transmission of raw personal chat content is planned. |
| **Reliability** | Retrieval accuracy is measured against a manually verified ground-truth query set (retrieval hit-rate and answer correctness), with systematic categorization of failure modes (bad chunk boundary, lost speaker context, wrong time window, irrelevant retrieval). |
| **Maintainability** | The pipeline is built as discrete, independently testable stages (parsing → chunking → embedding → retrieval), each with its own script and output format, so that chunking or retrieval strategies can be swapped without rewriting the rest of the pipeline. |
| **Privacy** | Personal conversational data is treated as a first-class design constraint, not an afterthought: existing RAG solutions typically require sending data to external/cloud models, whereas this project scopes toward local/on-device processing to keep chat content on the user's own device. |
| **Ethical Considerations (If Applicable)** | Chat data used for development and evaluation is either self-collected with the consent of team members and their contacts, or drawn from public research datasets; no chat content is shared beyond what is needed for the project. |

## 11 Expected Outcome

The project is expected to deliver:

1. A **comparative study report** diagnosing where and why existing RAG frameworks (LangChain, LlamaIndex) and memory architectures (e.g., MemGPT-style, segment-level memory) fail on fragmented, multi-party, time-dependent conversational data.
2. A **custom retrieval pipeline** for WhatsApp-style chat data, incorporating thread/reply-aware chunking, speaker-aware context, and time-weighted ranking, adapted specifically for conversational structure.
3. An **evaluation benchmark** – a labeled natural-language query set with manually verified ground-truth answers – to support reproducible retrieval-QA evaluation on personal chat data, since no standard public benchmark currently exists for this task.
4. A **working prototype assistant** (stretch goal, time permitting) capable of answering natural-language queries over a user's own WhatsApp history, with on-device processing to preserve privacy.
5. A **final technical/project report** consolidating the diagnostic findings, the adapted retrieval design, and the comparative evaluation results.

The core contribution is diagnostic and evidence-based rather than a new retrieval algorithm: identifying, through empirical benchmarking, exactly where generic RAG frameworks break down on multi-party human conversation, and demonstrating measurable improvement with a retrieval approach adapted to that setting.

## 12 References (Optional)

1. C. Pang et al., "SECOM: Towards effective memory granularity for retrieval-augmented response generation in long-term conversations," in *Proc. ICLR*, 2025.
2. C. Packer et al., "MemGPT: Towards LLMs as operating systems," *arXiv:2310.08560*, 2023.
3. N. Alonso et al., "Toward conversational agents with context and time sensitive long-term memory," *arXiv:2406.00057*, 2024.

4. B. J. Gutiérrez et al., "From RAG to memory: Non-parametric continual learning for large language models," *arXiv:2502.14802*, 2025.

5. Y. Tang and Y. Yang, "MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries," *arXiv:2401.15391*, 2024.

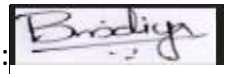
6. Infobip, "WhatsApp statistics 2026: Global usage & market overview," Infobip Blog, 2026.

7. Meta Platforms, Inc., "How to export your chat history," WhatsApp Help Center.

## Faculty Remarks

**Guide Remarks**

---

| **Project Guide** | **Project Coordinator** |
|---|---|
| Name: Bindhya Bhadran | Name: _____ |
| Signature:  | Signature: _____ |

# A Instructions to Students

## General Guidelines

- Complete all sections of the report before submission.
- Present technical content in a clear, concise, and professional manner.
- Use original work supported by appropriate references and citations.
- AI-assisted tools may be used only as supporting aids. Students are responsible for ensuring the originality, technical accuracy, and completeness of all submitted content.
- Reports containing unverified, generic, or AI-generated content without substantial student contribution may be rejected.
- All figures, tables, and diagrams must be clearly labeled and referenced in the text.
- Remove all placeholder text before final submission.

## Section-wise Guidelines

**Project Title** Provide a clear and meaningful title that reflects the proposed work.

**SDG Mapping** Identify the primary SDG and justify its relevance to the proposed project.

**Project Timeline** Summarize the major milestones, deliverables, and expected completion schedule.

**System Architecture**
Present the overall system architecture with a brief description of the major components.

**Module Description**
Describe the functionality of each module and its role in the system.

**System Workflow**
Illustrate the operational workflow using an appropriate diagram.

**Domain-Specific Design**
Include only the design artifacts relevant to the project domain.

**Proposed Solution**

Describe the proposed methodology or novel approach, highlighting its key contribution.

**Technology Stack**

List the programming languages, frameworks, tools, libraries, and platforms used.

**Requirements** Specify the hardware and software required for development and deployment.

**Design Considerations**

Discuss relevant aspects such as performance, scalability, security, maintainability, and reliability.

**Expected Outcome**

Summarize the expected deliverables and anticipated project outcomes.

**References** Cite all referenced literature using IEEE style.

## Submission Checklist

- All required sections are completed.
- Figures, tables, and references are properly numbered and cited.
- Grammar and formatting have been reviewed.
- The report has been verified and approved by the project guide prior to submission.